High-Performance APIs

A Senior Architectural Guide to ASP.NET Core Performance

Covering Caching, Rate Limiting, Compression & Timeouts

Introduction: The 3 Pillars of API Performance

In enterprise backend engineering, scaling a system to handle millions of requests is not just about writing "faster code" (like using `ValueTask`). It is about architectural defense mechanisms. High-performance APIs are built on three pillars:

1. **Avoid Work:** If you've calculated an answer once, don't do it again (Caching).
2. **Limit Work:** Do not let one greedy client starve the rest of the system (Rate Limiting & Timeouts).
3. **Shrink Work:** Make the data traveling over the network as small as possible (Compression).

1. Avoid Work: The ASP.NET Core Caching Hierarchy

Caching is the most effective way to improve performance. ASP.NET Core provides multiple layers of caching, from deep inside the memory to the edge of the HTTP pipeline.

Level 1: Object Caching (IMemoryCache & IDistributedCache)

This is used to cache raw C# objects (like Lists or Database results) inside your services.

- **In-Memory Cache:** Fast, stores objects directly in the server's RAM. *Problem:* In a Web Farm with 3 servers, Server A's cache doesn't exist on Server B.
- **Distributed Cache (Redis/SQL):** Stores serialized data in a central server. Slightly slower due to network travel, but guarantees consistency across a Web Farm.
- **Hybrid Cache (.NET 9+):** The holy grail. It automatically checks the local In-Memory cache (L1) first. If it misses, it checks the Distributed Cache (L2). It completely handles the serialization and synchronization automatically.

Level 2: The HTTP Boundary (Output vs. Response Caching)

This is a classic senior interview topic. What is the difference between Response Caching and Output Caching?

> **Response Caching (The Old Way)**
>
> Relies on HTTP Headers (`Cache-Control`). The server simply tells the client's browser or an intermediate proxy (like Cloudflare): *"Save this JSON for 60 seconds."* If the browser respects the header, it won't ask the server again. **Problem:** You have no control over it once it leaves your server. The client can ignore the header and hit your API anyway.

Output Caching (The Modern Way / .NET 7+)

A server-side middleware that caches the *entire, fully-generated HTTP Response* directly in your server's memory. When a request hits the pipeline, if the URL is cached, it instantly returns the cached JSON. **It bypasses the Controller, the Model Binding, and the Database entirely.**

```
// Modern Output Caching in Minimal APIs
app.MapGet("/api/products", async (dbContext) =>
{
    return await dbContext.Products.ToListAsync();
}).CacheOutput(policy => policy.Expire(TimeSpan.FromSeconds(60)));
```

2. Limit Work: Rate Limiting & Timeouts

When an API becomes successful, it becomes a target for abuse—either malicious (DDoS) or accidental (a client stuck in an infinite `while` loop calling your endpoint). You must implement pipeline constraints.

Rate Limiting Middleware (.NET 7+)

Rate limiting restricts how many HTTP requests a specific client (usually identified by IP Address or JWT User ID) can make.

Algorithm	How it Works	Best Use Case
Fixed Window	Allows 100 requests per 60 seconds. At exactly 60s, the counter resets to 0.	Simple API tiers (Basic vs Pro users).
Sliding Window	Similar to fixed, but splits the minute into segments. Prevents 100 requests at 0:59 and another 100 at 1:00.	Strict, smooth traffic shaping to protect databases.
Token Bucket	You have a bucket of 100 tokens. Every request takes 1 token. The bucket refills at a steady rate of 10 tokens/min.	APIs where users usually idle, but need sudden "bursts" of activity.
Concurrency Limiter	Does not care about time. Only allows exactly 10 requests to execute *at the exact same time*.	Protecting extremely heavy, CPU-bound machine learning or reporting endpoints.

Request Timeouts (.NET 8+)

By default, ASP.NET Core will let a request run forever. If your database locks up, 1,000 incoming requests will consume 1,000 server threads, waiting indefinitely. Within seconds, your API will suffer **Thread Pool Starvation** and crash.

The `UseRequestTimeouts()` middleware allows you to set a hard global or endpoint-specific limit (e.g., 5 seconds). If the endpoint hasn't finished, the framework triggers the `CancellationToken` and returns a `504 Gateway Timeout`.

> **CRITICAL REQUIREMENT:** Timeouts only work if you actually pass the `CancellationToken` down into your Entity Framework Core queries (e.g., `await _db.Users.ToListAsync(ct)`)! If you don't pass the token, EF Core will keep querying the database even after the HTTP response has timed out!

3. Shrink Work: Response Compression

If your API returns a massive array of 5,000 JSON products, that could easily be a 2MB payload. Sending 2MB over a mobile 3G connection is incredibly slow.

The `UseResponseCompression()` middleware intercepts the outgoing JSON, compresses it using algorithms like **Brotli (fastest/smallest)** or **Gzip**, and sends a tiny 300KB payload to the client. The client's browser/HTTP client automatically decompresses it.

```
// In Program.cs
builder.Services.AddResponseCompression(options =>
{
    options.EnableForHttps = true; // Required for modern APIs
    options.Providers.Add<BrotliCompressionProvider>();
});

// Add to pipeline (Must be BEFORE caching so you cache the compressed version!)
app.UseResponseCompression();
```

> **SECURITY WARNING (CRIME/BREACH Attacks):**
> Enabling compression over HTTPS was historically considered dangerous if the response contained *user-controlled input* mixed with *secret data* (like an Anti-Forgery token inside an HTML page), because hackers could guess the secret by measuring the compressed payload size. However, for pure REST APIs returning standard JSON arrays, compressing over HTTPS is standard practice.